

8. Bagging and random forests

25.11.2025

ETH zürich

D MATH

Outline

- ➊ Motivation: Why use ensemble methods?
- ➋ Bootstrap (parametric and nonparametric bootstrap) and aggregating: Bagging
- ➌ Random forests

Expected time: ~90 minutes

What are ensemble methods?

- Ensemble learning (also "model aggregation", "model averaging", "mixture models") are methods which combine various prediction functions $\hat{f}^{(1)}, \dots, \hat{f}^{(K)}$ into a single $\hat{f}$.
- Basic examples:

Average (Regression) :

$$\hat{f}(x) = \frac{1}{K} \sum_{k=1}^K \hat{f}^{(k)}(x)$$

Majority vote (Classification) :

$$\hat{f}(x) \in \arg \max_{y \in \mathcal{Y}} \sum_{k=1}^K \mathbf{1}_{\hat{f}^{(k)}(x)=y}$$

Mixture model:

$$\hat{f}(x) = \sum_{k=1}^K \hat{f}^{(k)}(x) \mathbf{1}_{\mathcal{X}_k}(x), \quad \mathcal{X} = \bigcup_{k=1}^K \mathcal{X}_k$$

- Ensemble learning techniques started around the late 1980s (cf. Clemen, 1989), and are especially useful and still frequently used together with trees, like in Bagging (Breiman, 1996) and Boosting (Schapire, 1990). We will mainly refer to the book by Hastie et al., 2009
- Oftentimes, ensemble learning methods rely on resampling techniques: In this regard, we will look at bootstrapping

Benefits of ensemble methods

- The intuitive benefits of ensemble methods are loosely related to diversification in finance: If several estimators are available, then diversification across those estimators is less risky.
- The "risk" is hereby, for instance, the variability of the estimator if different data is used
- This intuition rests on an important assumption: The estimators $\hat{f}^{(1)}, \dots, \hat{f}^{(K)}$ are **not** strongly positively correlated (making sure this is the case is a key part of random forests)
- While this lecture focuses mostly on tree methods, we emphasize that ensemble methods can also be useful for neural networks (see Opitz and Maclin, 1999 for an early empirical study). In this regard, it is for instance also rumored that GPT-4 is a mixture model ("mixture of experts") of 8 different 220 billion parameter models

Bootstrapping: Setting

Let $Z_1, Z_2, \dots, Z_m \sim \rho \in \mathcal{P}(\mathcal{Z})$ be iid and $\hat{\theta}$ an estimator of some unknown parameter θ ,

$$\hat{\theta} = g(Z_1, Z_2, \dots, Z_m),$$

where g is a function obtaining this estimator from data $Z_1, Z_2, \dots, Z_m$.

Example 8.1

Sample mean: $\hat{\theta} = \frac{1}{m} \sum_{i=1}^m Z_i$

Poly. ridge regr.: $\hat{\theta} = \arg \min_{\alpha \in \mathbb{R}^p} \frac{1}{m} \sum_{i=1}^m \left(Y_i - \sum_{j=0}^{p-1} \alpha_{j+1} (X_i)^j \right)^2 + \lambda \|\alpha\|_2^2, \quad (\text{where } Z_i = (X_i, Y_i))$

Question: What is the distribution of $\hat{\theta}$?

→ Needed, for instance, for statistical inference (confidence intervals, significance testing, etc).

E.g., for the sample mean with $\mathcal{Z} = \mathbb{R}$, one may want to know $\mathbb{P}(\hat{\theta} \in [a, b])$

Bootstrapping: Goal and basic idea

$$\hat{\theta} = g(Z_1, Z_2, \dots, Z_m), \quad Z_1, \dots, Z_m \sim \rho \text{ iid}$$

Question: What is the distribution of $\hat{\theta}$?

Answer: Unless we know ρ and g and both are tractable, then **we don't know**.

Bootstrapping: The method of bootstrapping, introduced by Efron, 1979, is a way to **approximate** the distribution of $\hat{\theta}$ only using $Z_1, \dots, Z_m$.

Basic idea of bootstrapping: Let us call $\kappa = \mathbb{P} \circ \hat{\theta}^{-1}$ the distribution of $\hat{\theta}$.

- Suppose we could simulate iid copies $(Z_1^{(k)}, \dots, Z_m^{(k)})$ of $(Z_1, \dots, Z_m)$, $k = 1, \dots, K$, and set $\hat{\theta}^{(k)} := g(Z_1^{(k)}, \dots, Z_m^{(k)})$. Then $\frac{1}{K} \sum_{k=1}^K \delta_{\hat{\theta}^{(k)}}$ converges to κ (w.r.t. weak convergence)
→ Impossible, because we don't know ρ !
- But we can approximate ρ via the empirical measure $\hat{\rho}_m := \frac{1}{m} \sum_{i=1}^m \delta_{Z_i}$ (which converges to ρ for $m \rightarrow \infty$ w.r.t. weak convergence)
- Thus we can simulate $\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)} \sim \hat{\rho}_m$ iid, $k = 1, \dots, K$, and set $\hat{\theta}^{(k)} := g(\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)})$. Then $\kappa^{(m,K)} := \frac{1}{K} \sum_{k=1}^K \delta_{\hat{\theta}^{(k)}}$ is the **bootstrap approximation of κ**

Bootstrapping: Non-parametric bootstrap algorithm

$$\hat{\theta} = g(Z_1, Z_2, \dots, Z_m), \quad Z_1, \dots, Z_m \sim \rho \text{ iid}$$

Algorithm Non-parametric bootstrap algorithm

- 1: Set $K \in \mathbb{N}$.
 - 2: **for** $k = 1, \dots, K$ **do**
 - 3: Take m uniformly random drawings $\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)}$ with replacement from $(Z_1, \dots, Z_m)$
 {iid sample from $\hat{\rho}_m$ }
 - 4: $\hat{\theta}^{(k)} := g(\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)})$
 - 5: **end for**
 - 6: Return $(\hat{\theta}^{(1)}, \dots, \hat{\theta}^{(K)})$ {encoding the bootstrap distribution $\kappa^{(m,K)} = \frac{1}{K} \sum_{k=1}^K \delta_{\hat{\theta}^{(k)}}$ }
-

For example, one can use the bootstrap distribution to approximate probabilities for confidence intervals: (for simplicity, $\mathcal{Z} = \mathbb{R}$)

$$\mathbb{P} \left(\hat{\theta} \in [a, b] \right) = \kappa([a, b]) \approx \kappa^{(m,K)}([a, b]) = \frac{1}{K} \sum_{k=1}^K \mathbf{1}_{[a,b]}(\hat{\theta}^{(k)})$$

Bootstrapping: Parametric bootstrap algorithm

$$\hat{\theta} = g(Z_1, Z_2, \dots, Z_m), \quad Z_1, \dots, Z_m \sim \rho \text{ iid}$$

Parametric bootstrapping: We have **additional information** regarding the distributional form of ρ , say $\rho = \lambda(\xi)$ for some unknown parameter ξ . For example, $\lambda(\xi) = \mathcal{N}(\mu, \sigma)$ with $\xi = (\mu, \sigma)$.

→ Estimation of ρ is usually easier since we only need to estimate ξ .

→ Say $\hat{\xi} = h(Z_1, Z_2, \dots, Z_m)$ is the estimator of ξ based on $Z_1, \dots, Z_m$.

Algorithm Parametric bootstrap algorithm

- 1: Set $K \in \mathbb{N}$.
 - 2: **for** $k = 1, \dots, K$ **do**
 - 3: Let $\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)}$ iid from $\lambda(\hat{\xi})$
 - 4: $\hat{\theta}^{(k)} := g(\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)})$
 - 5: **end for**
 - 6: Return $(\hat{\theta}^{(1)}, \dots, \hat{\theta}^{(K)})$ {encoding the bootstrap distribution $\kappa^{(m,K)} = \frac{1}{K} \sum_{k=1}^K \delta_{\hat{\theta}^{(k)}}$ }
-

Aggregating

To emphasize the potential benefit of aggregating, consider the **idealized setting** for

$$\textbf{poly. ridge regression: } \hat{\theta}^{(k)} = \arg \min_{\alpha \in \mathbb{R}^p} \frac{1}{m} \sum_{i=1}^m \left(Y_i^{(k)} - \sum_{j=0}^{p-1} \alpha_{j+1} \left(X_i^{(k)} \right)^j \right)^2 + \lambda \|\alpha\|_2^2,$$

where $(X_i^{(k)}, Y_i^{(k)}) \in \mathbb{R}^2, i = 1, \dots, m, k = 1, \dots, K$, are iid samples from ρ . For $\theta \in \mathbb{R}^p$, let

$$f_{\theta}(x) := \sum_{j=0}^{p-1} \theta_{j+1} x^{(j)} \quad \text{and} \quad \mathcal{E}(\theta) := \int (y - f_{\theta}(x))^2 \rho(dx, dy).$$

Lemma 8.2

Let $\theta^{agg} := \frac{1}{K} \sum_{k=1}^K \hat{\theta}^{(k)}$. Then $\mathbb{E}[\mathcal{E}(\theta^{agg})] \leq \frac{1}{K} \sum_{k=1}^K \mathbb{E}[\mathcal{E}(\hat{\theta}^{(k)})] = \mathbb{E}[\mathcal{E}(\hat{\theta}^{(1)})]$.

Note: Under suitable assumptions, this inequality is strict and holds also for more general learning problems

$$\hat{f}^{(k)} = \arg \min_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \ell \left(f(X_i^{(k)}), Y_i^{(k)} \right) + \lambda R(f).$$

Bagging aims at combining **Bootstrapping** with **Aggregating**

- The setting on the previous slide was idealized because we have $m \cdot K$ many iid samples
- If we only have m many training samples as usual, then bagging aims to generate $m \cdot K$ many bootstrapped training samples (in the non-parametric case, these are simply samples from $\hat{\rho}_m := \frac{1}{m} \sum_{i=1}^m \delta_{Z_i}$)
- These bootstrapped samples are only independent **conditional on** the training data $(Z_1, \dots, Z_m)$, and thus theoretically less beneficial than true iid samples
- Nevertheless, one finds that using bootstrapped samples often leads to practical improvements, particularly if one uses tailor-made estimation procedures (which is done in random forests!)

Bagging algorithm

We are given training data $Z_1, \dots, Z_m = (X_1, Y_1), \dots, (X_m, Y_m) \sim \rho$. Our goal is to approximately solve the learning problem

$$f^* = \arg \min_{f \in \mathcal{H}} \int \ell(f(x), y) \rho(dx, dy)$$

Algorithm Bagging algorithm (non-parametric)

- 1: Set $K \in \mathbb{N}$.
 - 2: **for** $k = 1, \dots, K$ **do**
 - 3: Take m uniformly random drawings $\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)}$ with replacement from $(Z_1, \dots, Z_m)$
 - 4: Take $\hat{f}^{(k)} := \arg \min_{f \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m \ell(f(\tilde{X}_i^{(k)}), \tilde{Y}_i^{(k)}) + \lambda R(f)$
 - 5: **end for**
 - 6: Return $\hat{f}^{agg}$ as an aggregation from $\hat{f}^{(1)}, \dots, \hat{f}^{(K)}$ {Averaging, majority vote, or similar}
-

Shortcoming: There may be very little variability in $\hat{f}^{(k)}$, particularly if m is large.

Random forest idea: For $\mathcal{H}$ being decision trees, adjust step (4:) and specify $\hat{f}^{(k)}$ directly via a specific algorithm, which includes additional sources of randomness to increase variability of $\hat{f}^{(1)}, \dots, \hat{f}^{(K)}$ and thus increase the effect of aggregation

Bagging: Subbagging

- Often, it can make sense to only draw $\tilde{m} < m$ many random samples $\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_{\tilde{m}}^{(k)}$ to determine $\hat{f}^{(k)}$
- This approach is called "Subbagging" (cf. Bühlmann, 2003)
- In certain cases it can be shown that $\tilde{m} = \lceil m/2 \rceil$ is equivalent in terms of approximation accuracy, and thus subbagging is sometimes computationally cheaper and yet yields estimators of equal quality as bagging (see Buja and Stuetzle, 2006 and Politis, Romano, and Wolf, 1999)
- **Takeaway message:** In practice, it is often a useful heuristic to do subbagging with $\tilde{m} = \lceil m/2 \rceil$

Random forests: Recap

We have d many features, $\mathcal{X} \subseteq \mathcal{X}_1 \times \dots \mathcal{X}_d$.

Recall the tree growing algorithm for CART

Algorithm Basic structure of tree growing algorithm

- 1: Initialize $T = \{()\}$ and $P_{()} = \mathcal{X}$.
 - 2: **while** Stopping criterion (*) is not reached **do**
 - 3: Choose a node $t \in \mathcal{T}$ ($t \in \{0, 1\}^l$), a variable $j \in \{1, \dots, d\}$ and a set $C_j \subseteq \mathcal{X}_j$ according to criterion (**)
 - 4: Set $T \leftarrow T \cup \{t0, t1\}$, where $t0 = (t_1, t_2, \dots, t_l, 0)$ and $t1 = (t_1, t_2, \dots, t_l, 1)$
 - 5: Set $P_{t0} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \in C_j\}$, $P_{t1} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \notin C_j\}$
 - 6: **end while**
 - 7: Compute the values $V = \{\hat{y}_t : t \in \mathcal{T}\}$; hereby, for all $t \in \mathcal{T}$, $\hat{y}_t$ is calculated solely using the training data $\{y_i : i \in \{1, \dots, m\}, x_i \in P_t\}$ according to criterion (***)
 - 8: Return (T, P, V)
-

Note: The data $(X_i, Y_i), i = 1, \dots, m$, influences the algorithm through the criteria (*), (**) and (***), as specified in Chapter 7 on CART.

Why using random forests?

Two main disadvantages of standardized binary splits in the CART construction:

- They only lead to piece-wise constant estimates (because we apply rectangular splits)
- They can be rather unstable under slight changes in the observations, i.e. a small change in an observation may lead to a different split, resulting in a completely different tree.

For these reasons, one aims at constructing a whole family (ensemble) of tree estimators which should become more stable under averaging (and aggregating).

- ⇒ **Bagging** directly for CART: the estimators $\hat{f}^{(k)}$ are too dependent, only small (if no) effect from aggregating.
- ⇒ **Random forests**: introduce additional variability by picking at each node/binary split a random subset of the features which are used to grow the tree.

Random forests

Recall the tree growing algorithm for CART, **adjusted** for **random forests**:

Algorithm 5 Basic structure of tree growing algorithm

- 1: Initialize $T = \{()\}$ and $P_{()} = \mathcal{X}$, **and a distribution $\mathcal{U}$ over subsets of $\{1, \dots, d\}$.**
 - 2: **while** Stopping criterion (*) is not reached **do**
 - 3: **Sample $U \subseteq \{1, \dots, d\}$ from $\mathcal{U}$**
 - 4: Choose a node $t \in \mathcal{T}$ ($t \in \{0, 1\}^l$), a variable $j \in U$ and a set $C_j \subseteq \mathcal{X}_j$ according to criterion (**)
 - 5: Set $T \leftarrow T \cup \{t0, t1\}$, where $t0 = (t_1, t_2, \dots, t_l, 0)$ and $t1 = (t_1, t_2, \dots, t_l, 1)$
 - 6: Set $P_{t0} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \in C_j\}$, $P_{t1} = P_t \cap \{x \in \mathcal{X} : x^{(j)} \notin C_j\}$
 - 7: **end while**
 - 8: Compute the values $V = \{\hat{y}_t : t \in \mathcal{T}\}$; hereby, for all $t \in \mathcal{T}$, $\hat{y}_t$ is calculated solely using the training data $\{y_i : i \in \{1, \dots, m\}, x_i \in P_t\}$ according to criterion (***)
 - 9: Return (T, P, V) **corresponding to the tree function f^{RF}**
-

Benefits: The random choice of the subsets U is an internal source of randomness of the algorithm and leads to variability in the outputs f^{RF} .

Random forests: Algorithm

Algorithm Random forest algorithm (non-parametric)

- 1: Set $K \in \mathbb{N}$ and specify distribution $\mathcal{U}$ for Algorithm 5.
 - 2: **for** $k = 1, \dots, K$ **do**
 - 3: Take m uniformly random drawings $\tilde{Z}_1^{(k)}, \dots, \tilde{Z}_m^{(k)}$ with replacement from $(Z_1, \dots, Z_m)$
 - 4: Take $\hat{f}^{(k)}$ as the output f^{RF} of Algorithm 5 with data $\tilde{Z}_1^{(k)} = (\tilde{X}_1^{(k)}, \tilde{Y}_1^{(k)}), \dots, \tilde{Z}_m^{(k)} = (\tilde{X}_m^{(k)}, \tilde{Y}_m^{(k)})$
 - 5: **end for**
 - 6: Return $\hat{f}^{agg}$ as an aggregation from $\hat{f}^{(1)}, \dots, \hat{f}^{(K)}$ {Averaging, majority vote, or similar}
-

Comments:

- Often, $\mathcal{U}$ is simply the uniform distribution on subsets $U \subseteq \{1, \dots, d\}$ of size $\underline{d} < d$, where $\underline{d}$ is a hyperparameter
- Similar modifications as for subagging can be applied
- Usually, the hyperparameter α to specify the stopping criterion of Algorithm 5 are chosen rather small (so that we obtain large trees). This is done since overfitting is less of an issue with the increased variability through $\mathcal{U}$ and the effects of aggregating

Conclusion

- Bootstrapping is a resampling technique which generates additional artificial training data sets
- Aggregating is a technique to combine various models/estimators into a single (more powerful) estimator
- Bagging combines Bootstrapping and Aggregating
- Random forests are a particular way of Bagging for trees, where the individual trees that are aggregated are purposefully made more diverse through randomization
- Implementations are for instance given in `sklearn.ensemble` (`RandomForestRegressor`; `RandomForestClassifier`; `BaggingRegressor`; `BaggingClassifier`)

List of references I

- Breiman, Leo (1996). “Bagging predictors”. In: *Machine learning* 24, pp. 123–140.
- Bühlmann, Peter Lukas (2003). “Bagging, subbagging and bragging for improving some prediction algorithms”. In: *Research report/Seminar für Statistik, Eidgenössische Technische Hochschule (ETH)*. Vol. 113. Seminar für Statistik, Eidgenössische Technische Hochschule (ETH), Zürich.
- Buja, Andreas and Werner Stuetzle (2006). “Observations on bagging”. In: *Statistica Sinica*, pp. 323–351.
- Clemen, Robert T (1989). “Combining forecasts: A review and annotated bibliography”. In: *International journal of forecasting* 5.4, pp. 559–583.
- Efron, B. (1979). “Bootstrap Methods: Another Look at the Jackknife”. In: *The Annals of Statistics* 7.1, pp. 1–26. DOI: 10.1214/aos/1176344552. URL: <https://doi.org/10.1214/aos/1176344552>.
- Hastie, Trevor et al. (2009). *The elements of statistical learning: data mining, inference, and prediction*. Vol. 2. Springer.
- Opitz, David and Richard Maclin (1999). “Popular ensemble methods: An empirical study”. In: *Journal of artificial intelligence research* 11, pp. 169–198.
- Politis, Dimitris N, Joseph P Romano, and Michael Wolf (1999). *Subsampling*. Springer Science & Business Media.

List of references II

Schapire, Robert E (1990). “The strength of weak learnability”. In: *Machine learning* 5, pp. 197–227.